

Interrupt Controller

Programmable priority and preemption

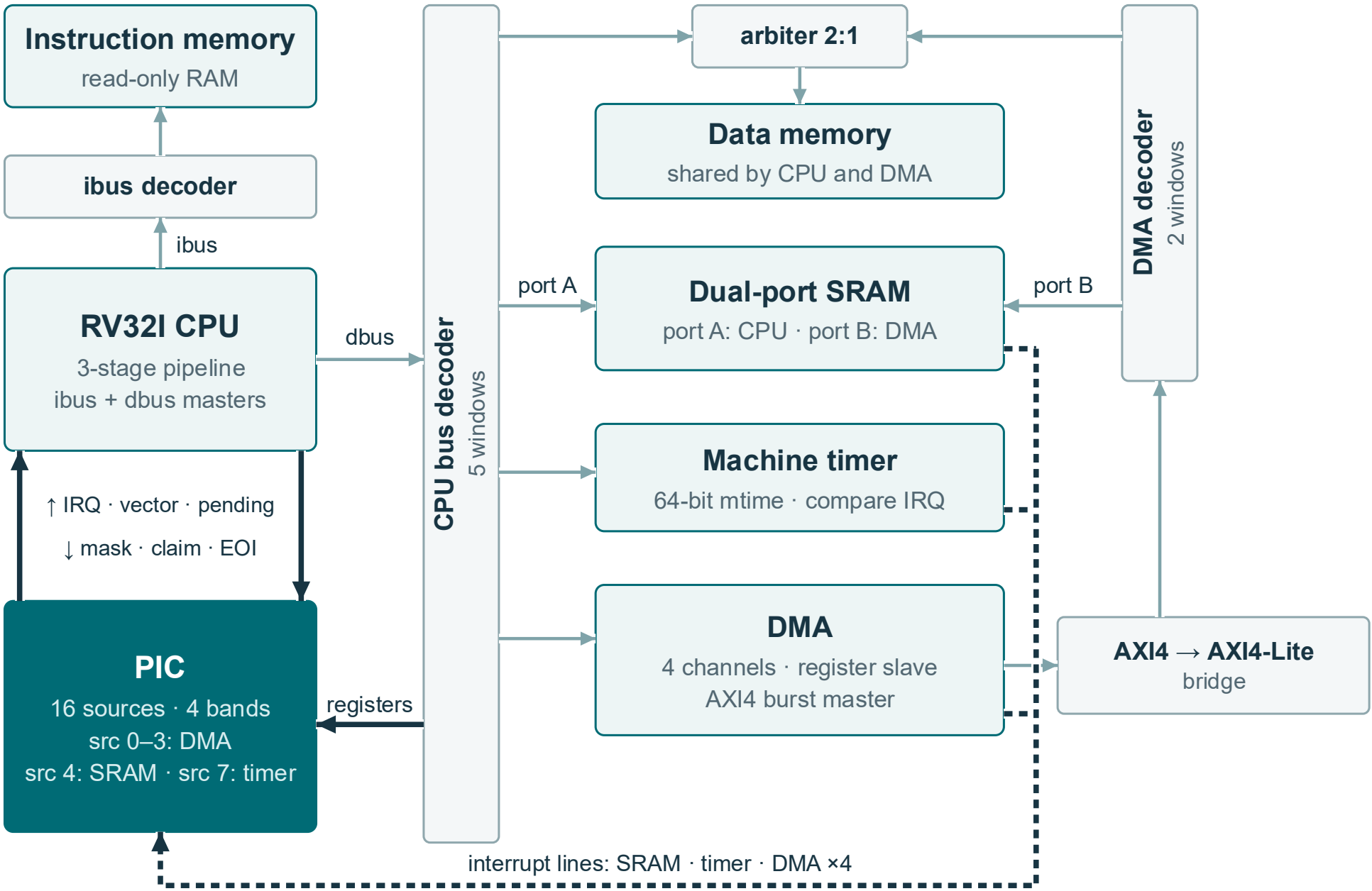
Bunea Cosmin-Andrei

Computer Engineering, year III

Academic year 2025–2026

Transilvania University of Braşov

RTL implementation · interfaces · verification



The brief asks for a PIC with priority bands, nesting and deadlines

01 16 hardware sources, each level- or edge-triggered

05 Spurious interrupts detected, handled and logged

02 16 software triggers, safe against accidental writes, arbitrated exactly like hardware

06 A deadline per source, with automatic priority escalation

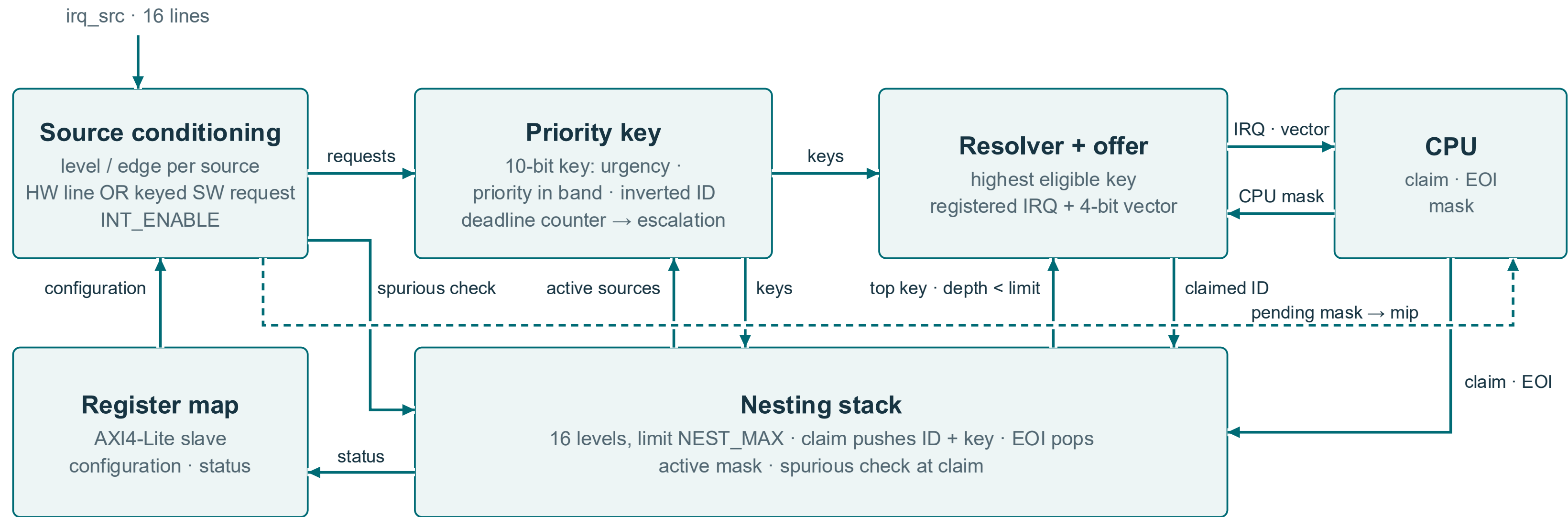
03 Priority bands, with arbitration inside each band

07 One request + 4-bit vector to the CPU, acknowledged by a claim

04 Preemption with nesting; the interrupted context is kept

08 AXI4-Lite register map: 11 registers, OKAY / SLVERR / DECERR

A request is conditioned, ranked by its key and offered to the CPU



THE BRIEF LEFT 21 QUESTIONS OPEN · THE FIVE THAT SHAPED THE CONTROLLER

Bands

4 bands, 2-bit urgency each (BAND_CONFIG). 10-bit key: urgency · priority in band · inverted ID, so the lower ID wins ties.

Reconfiguration

Claim pushes ID + key onto the stack. An active source keeps its saved key; a new band counts from its next request.

Nesting

16-entry stack, NEST_MAX 1–16 (reset 8). Only a strictly higher key preempts; at the limit no offer, overflow flag.

Spurious

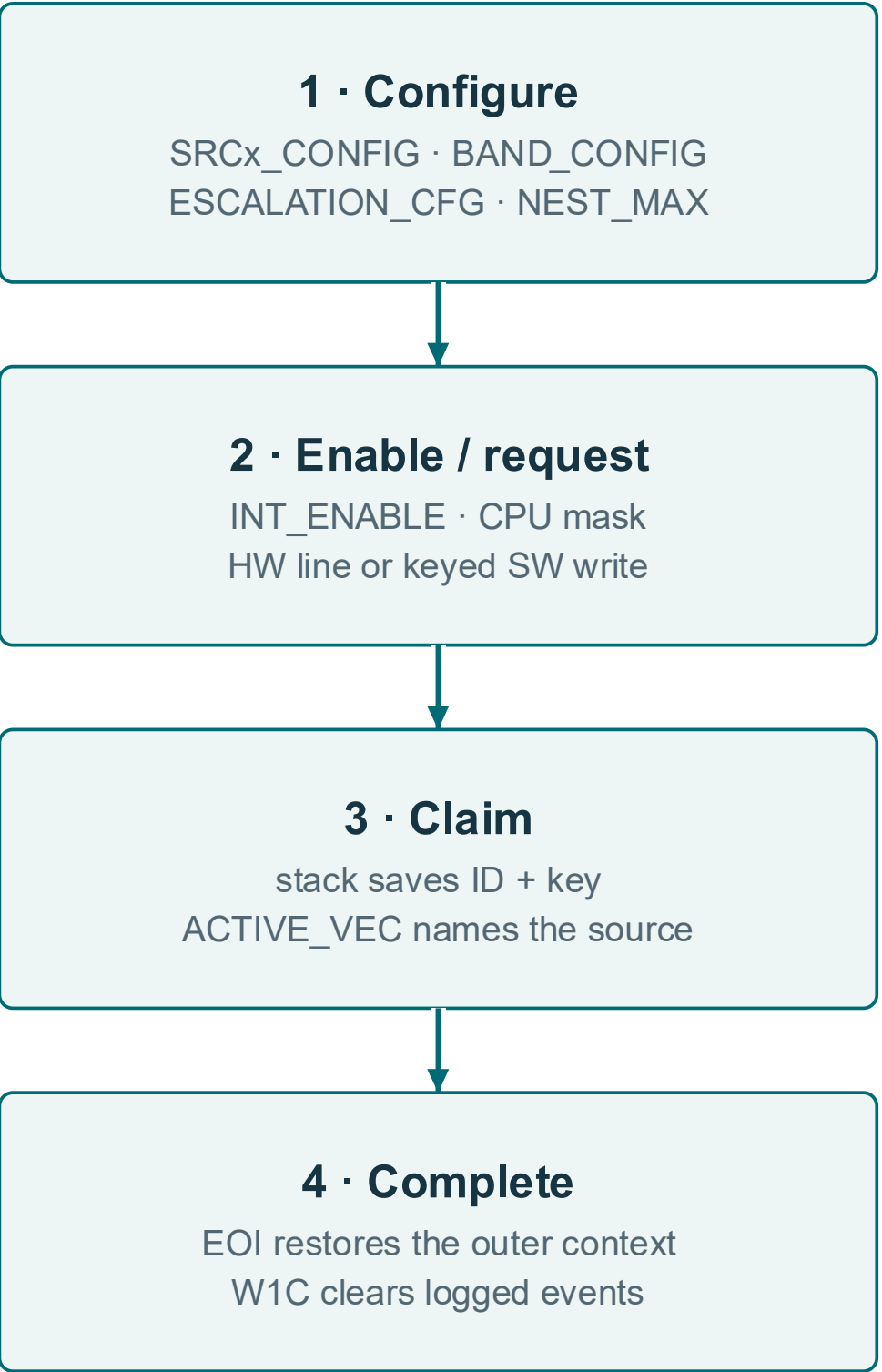
Detected at claim, when the request is gone. The level still opens and EOI closes it; logged in SPURIOUS_LOG.

Escalation

16-bit deadline counter per source. On a miss: jump to a target band or one urgency step up; optional repeat.

The registers set the policy and make the state observable

Register	What it does
SRCx_CONFIG ×16	Level / edge, band, priority in band, 16-bit deadline
SRCx_SW_TRIG ×16	Software request: key 0xA5A5 + both upper strobes; cleared by claim or write
SRCx_STATUS ×16	Pending, active, escalated, spurious, elapsed cycles, effective band
BAND_CONFIG	2-bit urgency for each of the 4 bands
ESCALATION_CFG	Jump to a target band or one step up; single or repeated
NEST_MAX	Nesting limit 1–16, reset 8; out-of-range writes clamped
INT_ENABLE	Per-source enable before arbitration
NEST_STATUS · ACTIVE_VEC	Depth, source on top, overflow · source in service
SPURIOUS_LOG · INT_STATUS	Sticky events, write 1 to clear; a hardware set wins



An independent model predicts every priority decision

THE PIC ALONE

- 275

golden-model cases: all 256 BAND_CONFIG values + 19 corners · 5,741 checks
- 15

random runs of 20,000 cycles, each cycle equal to a Python cycle model
- 171

feature and same-cycle checks: nesting, spurious, escalation, triggers
- 1,290

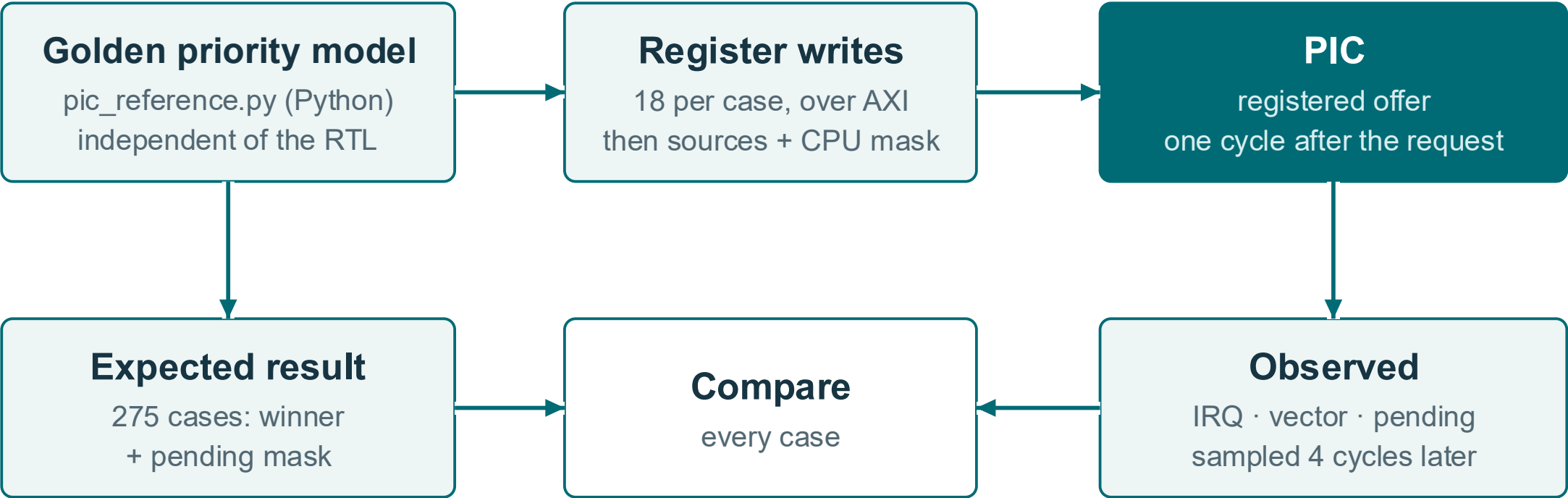
register checks: access rules 478 status 424 · reset 388
- 96

AXI4-Lite writes vs PULP axi_lite_regs: 16 WSTRB × 3 orders × 2 stalls
- 10

properties proven with SymbiYosys; 5 / 5 injected defects fail the proof
- 32

SVA properties attached with bind: 12 PIC, 20 AXI protocol
- 4 / 4

mutation testing: mask, tie-break, empty EOI, nesting limit



Case 72: BAND_CONFIG = 0x48 → urgency band 0: 0 · band 1: 2 · band 2: 0 · band 3: 1					
source	band	urgency	priority	CPU mask	result
0	0	0	5	on	
2	2	0	13	on	
9	1	2	7	off	pending, not offered
15	3	1	5	on	winner
model: offer 15 · pending 8205				PIC: offer 15 · pending 8205	
					✓

A missed deadline makes the CPU serve the escalated source first

THE PIC WITH THE CPU

382

checks, CPU + PIC + mtimer
program: priority, in-service
suppression, masked source

36

all 16 software-triggered sources
claimed by the CPU with the right
vector

70

nesting checks through the CPU: 2
levels (16) and all 16 levels (54)

15

spurious claim with the real CPU:
logged, EOI balanced, next source
served

12

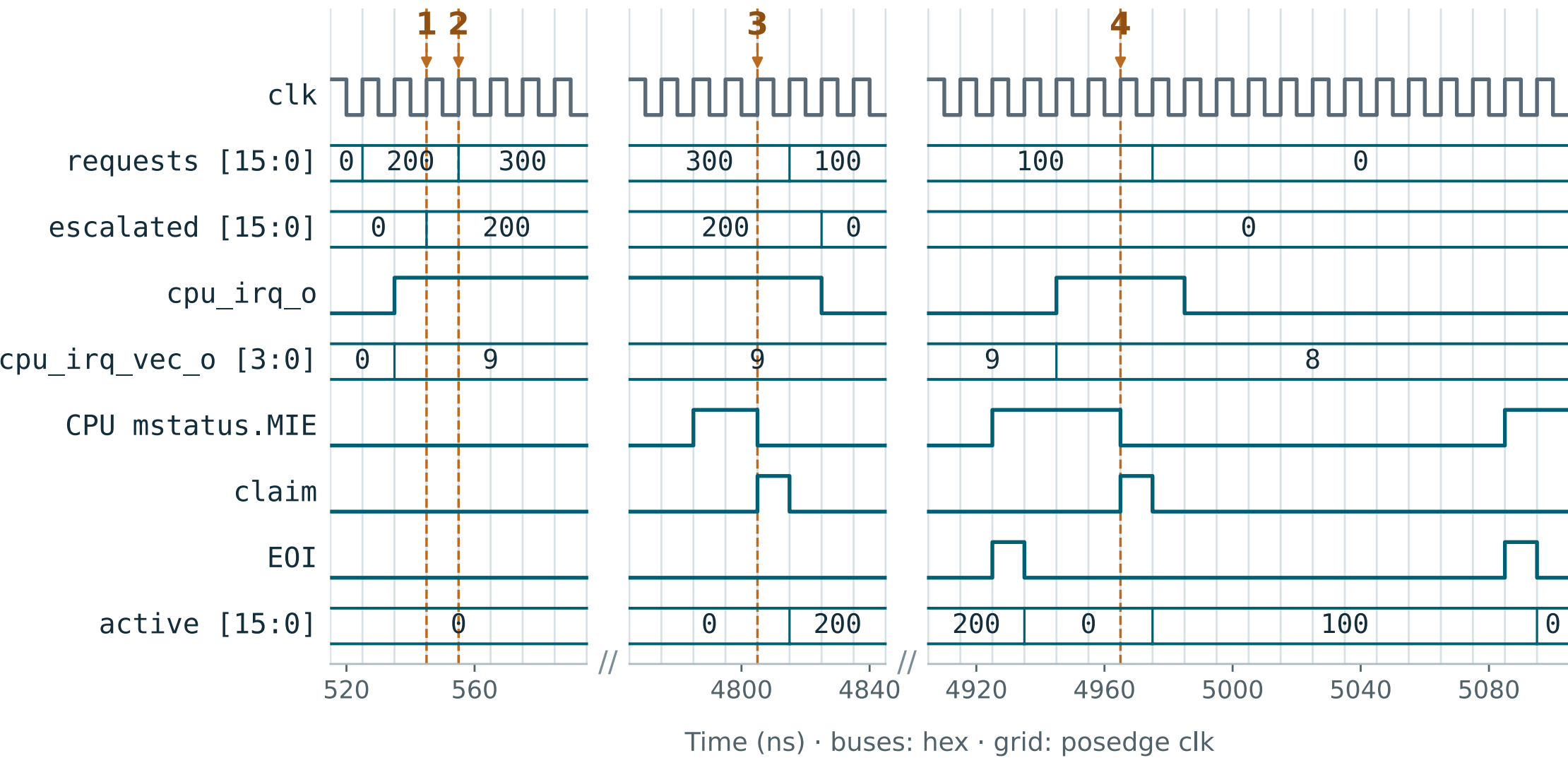
escalation through the CPU: service
order reversed

12

PIC SVA properties bound in all 23
SoC benches

CI

every push: lint, all PIC / CPU / SoC
benches with SVA, coverage and
cover gates



1 · Source 9 escalates

It has waited past its 2-cycle
deadline: escalated = 200
(bit 9), and it moves to band
0, the most urgent.

2 · Source 8 arrives

requests = 300 (8 and 9).
Band 1 would beat band 3,
but 9 is now in band 0, so
the vector stays 9.

3 · CPU takes source 9

The CPU sets MIE and claims
vector 9 first; active = 200
(source 9 in service).

4 · Then source 8

After the EOI of 9 the vector
changes to 8 and the CPU
claims it; active = 100 (bit 8).

Result: configurable arbitration with verified preemption

WHAT I BUILT

1. 4 bands with programmable urgency and a 10-bit priority key
2. 16-level nesting stack, spurious detection, deadline escalation
3. Golden priority model, cycle model and formal proofs of the stack

WHAT I LEARNED

- A priority encoder looks simple until two requests tie or arrive in the same cycle. I learned to write down a rule for every such case before touching the RTL.
- Saving the key at claim was a small idea with a big effect: software can reconfigure a source at any moment and the stack stays correct.
- Looking at the PIC and the CPU together showed me where the mask belongs: inside the PIC, so a source the CPU ignores never holds back the others.
- Writing the priority model in Python, apart from the Verilog, meant understanding every rule twice, and that is where the design became clear to me.

Thank you! Questions are welcome.